Politechnika Warszawska
Wydział Matematyki i Nauk Informacyjnych

Projekt 1: Klasyfikacja i regresja z użyciem wielowarstwowego perceptronu

Przedmiot: Sieci Neuronowe

Autorzy:
Jakub Rak
Krzysztof Radzewicz

Prowadzący:
[mgr inż. Mikołaj Małkiński]

Warszawa, 26 października 2026

Spis treści

1 Wstęp i cel projektu **1**

2 Implementacja wielowarstwowego perceptronu (MLP) **1**

3 Eksperymenty — Zagadnienie regresji **1**
3.1 Zbiory danych i metodyka badawcza 1
3.2 Wpływ funkcji aktywacji w warstwach ukrytych 3
3.3 Wpływ architektury sieci: głębokość i szerokość 4
3.4 Wpływ miary błędu (funkcji straty) 5
3.5 Wpływ rozmiaru zbioru danych (opcjonalnie) 7
3.6 Podsumowanie i wnioski z części regresyjnej 8

4 Eksperymenty — Zagadnienie klasyfikacji **9**
4.1 Zbiory danych 9
4.2 Eksperyment 1 - Funkcje aktywacji w warstwach ukrytych 10
4.2.1 Zbiór danych simple 10
4.2.2 Zbiór danych three_gauss 12
4.3 Eksperyment 2 - Architektura sieci 14
4.3.1 Zbiór danych simple 14
4.3.2 Zbiór danych three_gauss 16
4.4 Eksperyment 3 - Wpływ funkcji straty 18
4.4.1 Zbiór danych simple 18
4.4.2 Zbiór danych three_gauss 20
4.5 Podsumowanie i wnioski z części klasyfikacyjnej 23

5 Podsumowanie ogólne i wnioski końcowe **23**

1 Wstęp i cel projektu

Celem niniejszego projektu jest samodzielna implementacja wielowarstwowego perceptronu (*Multilayer Perceptron*, MLP) od podstaw z wykorzystaniem wyłącznie biblioteki `NumPy`. W ramach implementacji, przygotowano algorytm wstecznej propagacji błędu oraz stochastycznego spadku gradientu (*Stochastic Gradient Descent*). Przygotowany model posłużył do rozwiązania zagadnień aproksymacji nieliniowej dla funkcji $\mathbb{R} \to \mathbb{R}$ (zadanie regresji) oraz przydzielania klas punktom w przestrzeni $\mathbb{R}^2$ (zadanie klasyfikacji binarnej i wieloklasowej). W ramach pracy zbadano kilka aspektów wpływających na działanie sieci, takich jak dobór funkcji aktywacji (Sigmoid, Tanh, ReLU), głębokości i szerokości architektury (od 0 do 4 warstw ukrytych) oraz zastosowanych miar błędu na wyjściu modelu (MSE, BCE, CCE).

2 Implementacja wielowarstwowego perceptronu (MLP)

Kod źródłowy został podzielony na następujące moduły:

- **layers** — definiuje klasę bazową warstwy oraz warstwę w pełni połączoną (`Dense`), odpowiedzialną za transformację liniową, przechowywanie wag i biasów, a także inicjalizację wag przy pomocy zdefiniowanych rozkładów [1].
- **activations** — zawiera implementacje funkcji aktywacji (`ReLU`, `Sigmoid`, `Tanh`, i `Softmax`) wraz z metodami wyznaczania ich pochodnych na potrzeby propagacji wstecznej.
- **losses** — zawiera implementacje funkcji straty: błąd średniokwadratowy (`MSE`), błąd bezwzględny (`MAE`), binarna (`BCE`) i kategoryczna entropia krzyżowa (`CCE`).
- **optimizers** — implementuje algorytm *Stochastic gradient descent*.
- **mlp** — zamyka funckjonalność sieci w klasie `MLP`, przechowując listę warstw i implementując: propagację w przód, propagację wsteczną oraz kroki optymalizatora.
- **utils** — zawiera funkcje pomocnicze, w tym tworzenie mini batch'y, narzędzia do kodowania one-hot, metryki ewaluacyjne oraz ustawianie globalnego ziarna losowości.

Proces uczenia modelu sterowany jest przez metodę `MLP.fit()`, która w trakcie iteracji po kolejnych mini batchach i epokach na bieżąco zapisuje postępy uczenia. Wszystkie metadane zbierane są w formie słownika. Wśród jego kluczy znajdują się m.in. funkcje straty (w skali poszczególnych mini batchy oraz średnie na epokę dla zbioru treningowego i walidacyjnego) wprowadzone metryki (np. *Accuracy* dla klasyfikacji), oraz zmiany wartości kilku wybranych z góry wag i biasów w sieci.

3 Eksperymenty — Zagadnienie regresji

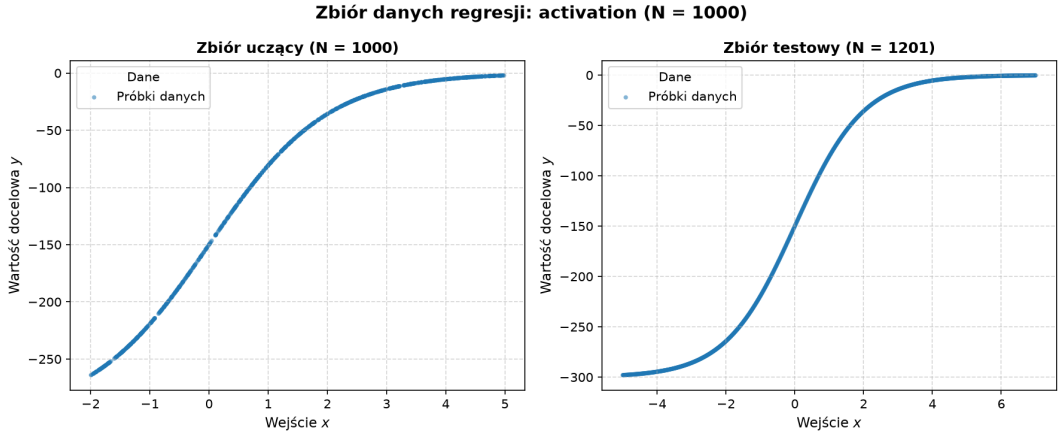
3.1 Zbiory danych i metodyka badawcza

Badania przeprowadzono dla dwóch jednowymiarowych zbiorów regresyjnych. W obu przypadkach wejście stanowi pojedyncza cecha ciągła, a zakres zbioru uczącego wynosi $x \in [-2, 5]$; model przewiduje jedną wartość rzeczywistą $\hat{y}$.

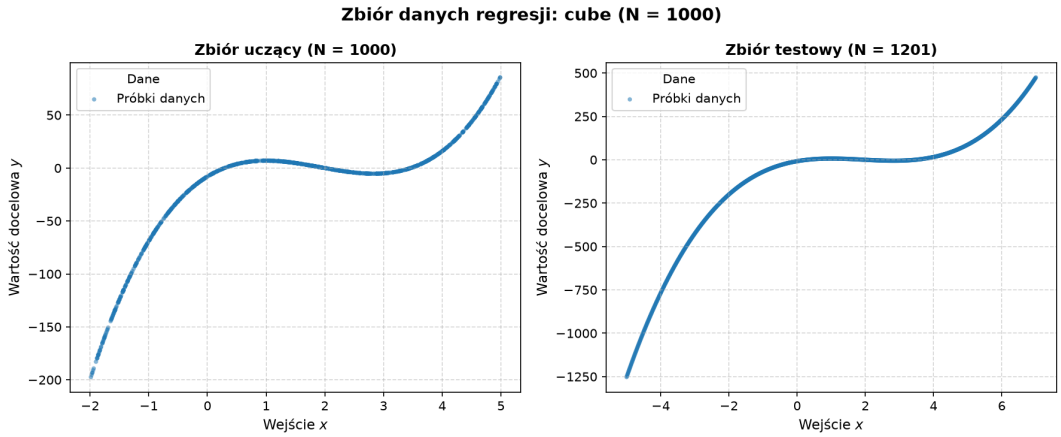
- **Zbiór `data.activation`** opisuje nieliniową funkcję o kształcie sigmoidalnym/przejściowym. Wartości celu w obszarze uczącym mieszczą się w przybliżeniu w zakresie $[-265, -2]$.
- **Zbiór `data.cube`** zawiera zależność wielomianową trzeciego stopnia z szumem, a wartości celu w obszarze uczącym znajdują się w przybliżeniu w zakresie $[-198, 86]$.

Wykorzystano gotowe podziały na zbiory uczące i testowe, dostępne dla liczności $N \in \{100, 500, 1000, 10000\}$. Dane w eksperymentach pozostawiono w oryginalnej skali — nie zastosowano standaryzacji ani normalizacji. Jest to istotne, ponieważ duży zakres wartości celu wpływa na skalę gradientów i wrażliwość uczenia na współczynnik η. Warstwa wyjściowa składała się z jednego neuronu z aktywacją tożsamościową (bez dodatkowej nieliniowości), dzięki czemu model mógł przewidywać wartości z dowolnego zakresu. Domyślnie stosowano bias. Jakość mierzono przez MSE, MAE i współczynnik determinacji R^2; wartości w tabelach są średnimi z trzech przebiegów dla ziaren 42, 123 i 999, o ile nie zaznaczono inaczej. Liczbę epok zwiększono do 1000 w tych eksperymentach, w których krzywe błędu nadal opadały przy poprzednim limicie. Wyjątkiem jest R1 dla `activation` (150 epok, gdzie strata szybko osiąga plateau) oraz R3 (150 epok, gdzie większe kroki uczenia wykazywały niestabilność).

Na rysunkach krzywe modelu naniesiono na punkty odpowiednich podzbiorów. W eksperymentach architektonicznych i dotyczących funkcji straty podział testowy obejmuje zakresy poza przedziałem uczącym, dlatego ocenia nie tylko interpolację, lecz także ekstrapolację. Szczególnie wyraźnie widać to dla `data.cube`, którego wielomian rośnie gwałtownie poza obszarem danych uczących.



Rysunek 1: Zbiory uczący i testowy dla `data.activation` ($N = 1000$).



Rysunek 2: Zbiory uczący i testowy dla `data.cube` ($N = 1000$). Widoczny jest wzrost wartości funkcji poza zakresem uczącym.

3.2 Wpływ funkcji aktywacji w warstwach ukrytych

W eksperymencie R1 porównano funkcje `Sigmoid`, `Tanh` i `ReLU` w architekturze $[1, 32, 16, 1]$. Warianty dla `data.activation` uczono przez 150 epok, natomiast dla `data.cube` liczbę epok zwiększono do 1000, ponieważ strata testowa nadal malała przy poprzednim limicie. W obu przypadkach stosowano mini batch o rozmiarze 32 i współczynnik uczenia $\eta = 0.001$. Dla `Sigmoid` i `Tanh` zastosowano inicjalizację Xavier, a dla `ReLU` — He.

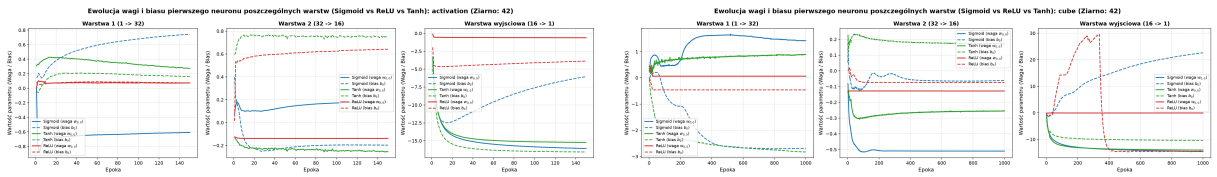
Tabela 1: Wyniki eksperymentu R1 na zbiorze testowym: średnia ± odchylenie standardowe z trzech ziaren.

Zbiór	Aktywacja	MSE	MAE	R^2
`activation`	Sigmoid	191.66 ± 1.42	7.87 ± 0.04	0.9867
	Tanh	199.12 ± 0.15	7.18 ± 0.12	0.9862
	ReLU	756.57 ± 235.07	15.62 ± 6.55	0.9476
`cube`	Sigmoid	76668.59 ± 488.48	137.26 ± 0.93	0.3672
	Tanh	77887.78 ± 1142.43	135.32 ± 1.58	0.3572
	ReLU	102872.31 ± 56930.40	177.66 ± 66.49	0.1510

Na `data.activation` wszystkie trzy warianty dobrze odtworzyły przebieg w obszarze uczącym. Na zbiorze testowym najlepszą wartość MSE uzyskał `Sigmoid` (191.66), a najniższe MAE — `Tanh` (7.18); wyniki tych dwóch gładkich funkcji były zbliżone. `ReLU` osiągnęła większy błąd i wyraźnie większą zmienność pomiędzy ziarnami. Krzywa `ReLU` jest odcinkami liniowa, natomiast `Sigmoid` i `Tanh` dają gładkie przejście. Różnice są szczególnie widoczne poza zakresem uczącym: aktywacje gładkie zbliżają się do stałych wartości, a `ReLU` kontynuuje ekstrapolację liniową.

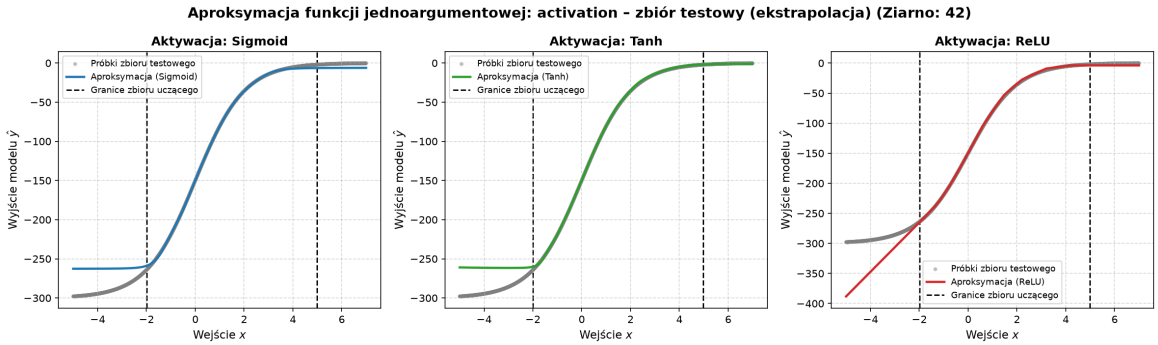
Wyniki dla `data.cube` po 1000 epokach nadal są słabe na zbiorze testowym, ponieważ podział wymaga ekstrapolacji szybko rosnącego wielomianu. Najniższy średni testowy MSE uzyskał `Sigmoid` (≈ 76669), niewiele niższy niż `Tanh` (≈ 77888). Wynik `ReLU` pogorszył się względem przebiegu 150-epokowego, a odchylenie standardowe wzrosło do około 56930; trajektorie wskazują na niestabilność części uruchomień. To przykład, że samo zwiększenie budżetu epok nie gwarantuje poprawy generalizacji. Aktywacje `Sigmoid` i `Tanh` spłaszczają predykcję poza zakresem uczącym, nie odwzorowując nieograniczonego wzrostu wielomianu.

Rysunek 3 przedstawia przebieg wagi $w_{0,0}$ (linia ciągła) i biasu b_0 (linia przerywana) pierwszego neuronu w każdej warstwie dla reprezentatywnego ziarna 42. W przypadku `cube` widoczna jest duża zmienność parametrów wariantu `ReLU`, zgodna z wysokim odchyleniem standardowym wyniku testowego po 1000 epokach.

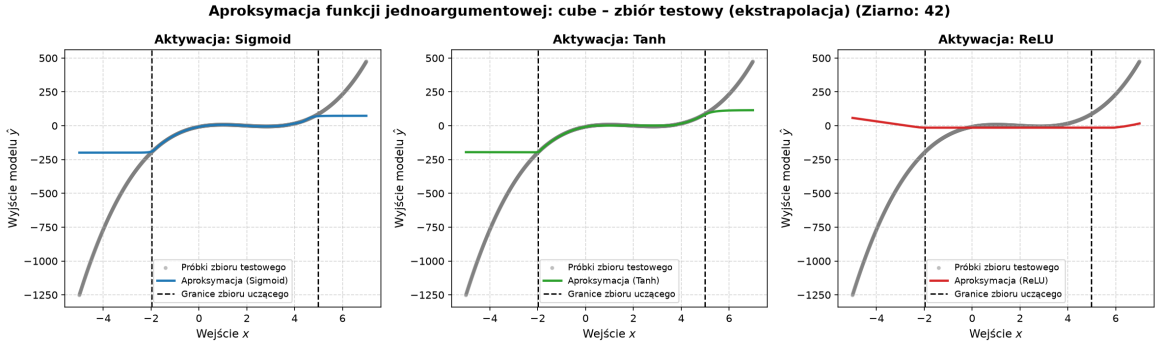


(a) `data.activation` (150 epok). (b) `data.cube` (1000 epok).

Rysunek 3: Ewolucja wagi wejściowej i biasu pierwszego neuronu w kolejnych warstwach sieci R1 dla funkcji aktywacji Sigmoid, Tanh i ReLU (ziarno 42).



Rysunek 4: Aproksymacja `data.activation` na zbiorze testowym dla trzech funkcji aktywacji. Linie przerywane wyznaczają zakres uczący; poza nim widoczne są odmienne zachowania ekstrapolacyjne.



Rysunek 5: Aproksymacja `data.cube` na zbiorze testowym dla trzech funkcji aktywacji. Żadna z krzywych nie odtwarza dobrze wzrostu wielomianu poza zakresem uczącym.

3.3 Wpływ architektury sieci: głębokość i szerokość

W eksperymencie R2 sprawdzono osiem architektur: model bez warstwy ukrytej $[1, 1]$, sieci z jedną warstwą o szerokości 4, 16 lub 64 neuronów, dwie wariacje z dwiema warstwami oraz sieci z trzema i czterema warstwami ukrytymi. Pozostałe warunki ustalono: aktywacja ukryta `Tanh`, inicjalizacja Xavier, MSE, mini batch 32, 1000 epok oraz $\eta = 0.0005$. Poniższe wartości to średnie testowe z trzech ziaren.

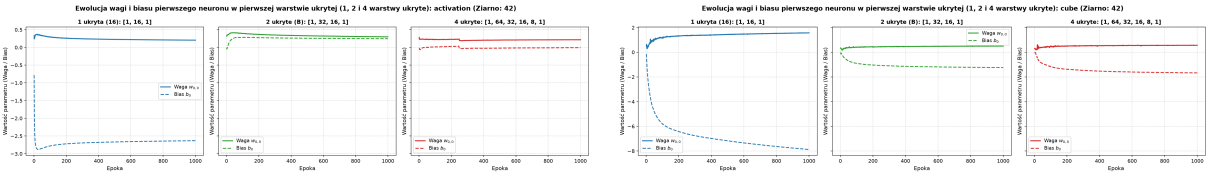
Tabela 2: Średni testowy MSE i R^2 w zależności od architektury (eksperyment R2, $N = 1000$).

	`data.activation`		`data.cube`	
Architektura	**MSE**	R^2	**MSE**	R^2
$[1, 1]$ (0 ukrytych)	2437.11	0.831	93426.16	0.229
$[1, 4, 1]$	210.89	0.985	81433.92	0.328
$[1, 16, 1]$	131.43	0.991	73277.09	0.395
$[1, 64, 1]$	45.31	0.997	65607.87	0.459
$[1, 16, 8, 1]$	178.02	0.988	78744.20	0.350
$[1, 32, 16, 1]$	145.61	0.990	76083.64	0.372
$[1, 32, 16, 8, 1]$	197.64	0.986	78432.35	0.353
$[1, 64, 32, 16, 8, 1]$	194.35	0.987	78639.15	0.351

Po wydłużeniu treningu do 1000 epok najlepszy w obu zbiorach był szeroki, jednowarstwowy model $[1, 64, 1]$: uzyskał MSE 45.31 dla `data.activation` i 65607.87 dla `data.cube`. Dodatkowa

głębokość nie poprawiła wyników w porównaniu z tym wariantem. Wartości dla `cube` pozostają wysokie ze względu na ekstrapolację.

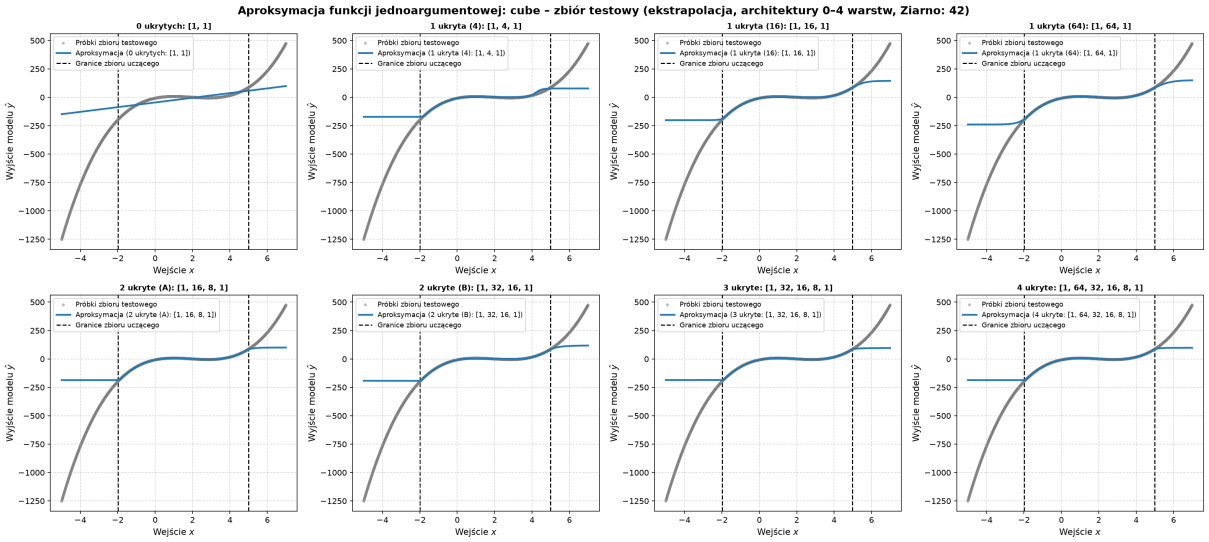
Na Rysunku 6 pokazano zmianę $w_{0,0}$ i b_0 pierwszego neuronu warstwy ukrytej dla sieci z jedną, dwiema i czterema warstwami ukrytymi. Krzywe dotyczą ziarna 42; porównanie parametrów ma charakter ilustracyjny, ponieważ neuron o tym samym indeksie w różnych architekturach nie musi pełnić tej samej funkcji.



(a) `data.activation`. (b) `data.cube`.

Rysunek 6: Ewolucja wagi $w_{0,0}$ i biasu b_0 pierwszego neuronu pierwszej warstwy ukrytej dla architektur o różnej głębokości w eksperymencie R2 (ziarno 42, 1000 epok).

Wartości treningowe były znacznie niższe od testowych, szczególnie dla `data.cube`. Rozbieżność ta jest zgodna z faktem, że test ocenia ekstrapolację, a nie wyłącznie interpolację. Nie należy jej zatem interpretować wyłącznie jako klasycznego przeuczenia do punktów treningowych. Przykładowe krzywe predykcji dla wszystkich architektur na zbiorze testowym przedstawia Rysunek 7.



Rysunek 7: Wpływ głębokości i szerokości sieci na ekstrapolację `data.cube` (reprezentatywne ziarno 42).

3.4 Wpływ miary błędu (funkcji straty)

W eksperymencie R3 porównano minimalizację MSE i MAE w tej samej architekturze $[1, 32, 16, 1]$ z aktywacją `Tanh`. Ponieważ MAE ma gradient o w przybliżeniu stałym module, a gradient MSE zależy od wielkości reszty, dla obu funkcji sprawdzono trzy współczynniki uczenia (0.001, 0.005 i 0.01). Każdy wariant trenowano przez 150 epok. Końcowe MSE i MAE obliczano tak samo niezależnie od optymalizowanej funkcji straty.

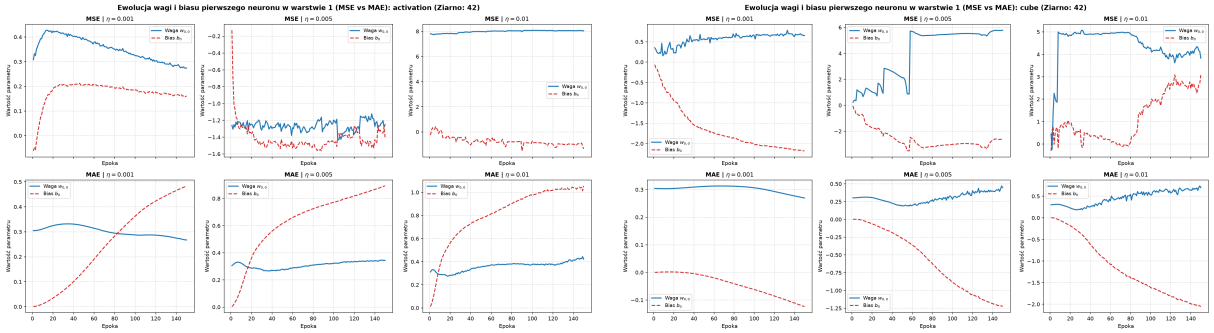
Tabela 3: Porównanie najlepszych testowych konfiguracji MSE i MAE według testowego MSE (średnia z trzech ziaren).

Zbiór	**Minimalizowana strata**	η	**Testowy MSE**	**Testowy MAE**
`activation`	MSE	0.001	199.12	7.18
`activation`	MAE	0.010	878.20	17.57
`cube`	MSE	0.010	81926.54	160.09
`cube`	MAE	0.010	97470.71	161.91

W badanym zakresie parametrów optymalizacja MSE dała najlepsze wyniki na obu zbiorach. Dla `data.activation` MSE przy $\eta = 0.001$ osiągnęło testowy MSE 199.12, podczas gdy najlepszy wariant MAE (według tego samego kryterium) uzyskał 878.20. Dla `data.cube` MSE przy $\eta = 0.01$ uzyskało testowy MSE 81926.54, a najlepszy wariant MAE — 97470.71. Wysokie błędy `cube` przypominają, że podział testowy obejmuje ekstrapolację.

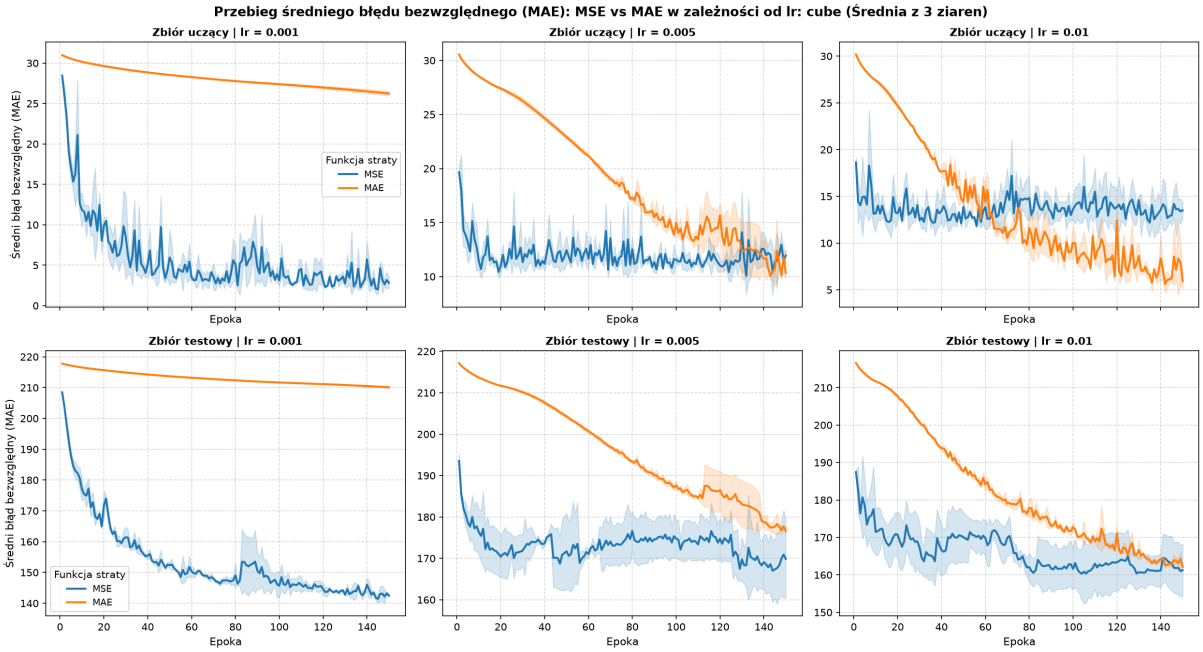
Krzywe uczenia pokazują, że porównanie zależy od kroku uczenia. Przy małym η MAE poprawiało się powoli, natomiast większy krok przyspieszał spadek błędu bezwzględnego. MSE reagowało silniej na duże reszty, lecz większe kroki powodowały niestabilniejsze przebiegi i gorsze wyniki końcowe. W otoczeniu minimum stały moduł gradientu MAE może utrudniać dokładne ustalenie parametrów i prowadzić do oscylacji; MSE daje coraz mniejsze korekty wraz ze zmniejszaniem reszty. Wyniki te dotyczą przetestowanych konfiguracji i nie oznaczają, że MAE jest ogólnie gorszą miarą — funkcja straty i tempo uczenia powinny być dobierane wspólnie.

Rysunek 8 pokazuje ewolucję wagi $w_{0,0}$ i biasu b_0 pierwszego neuronu w pierwszej warstwie ukrytej. Osobne panele odpowiadają optymalizowanej stracie i wartości współczynnika uczenia; przedstawiono reprezentatywne ziarno 42.



(a) `data.activation`. (b) `data.cube`.

Rysunek 8: Trajektorie wagi $w_{0,0}$ i biasu b_0 podczas optymalizacji MSE i MAE dla trzech wartości współczynnika uczenia (R3, ziarno 42, 150 epok).



Rysunek 9: Przebieg MAE na zbiorach uczącym i testowym dla optymalizacji MSE oraz MAE na `data.cube`, przy trzech wartościach współczynnika uczenia.

3.5 Wpływ rozmiaru zbioru danych (opcjonalnie)

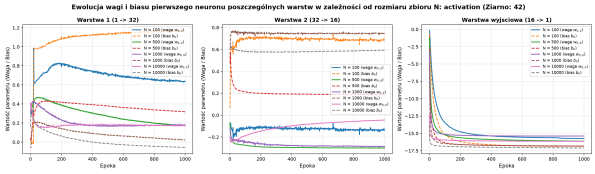
W eksperymencie R4 zbadano wpływ liczności zbioru uczącego $N \in \{100, 500, 1000, 10000\}$. Dla każdej liczności użyto architektury $[1, 32, 16, 1]$, aktywacji `Tanh`, inicjalizacji Xavier, straty MSE, $\eta = 0.001$ i 1000 epok. Wyniki są średnimi z trzech ziaren.

Tabela 4: Testowy MSE w zależności od liczności zbioru uczącego (średnia ± odchylenie standardowe).

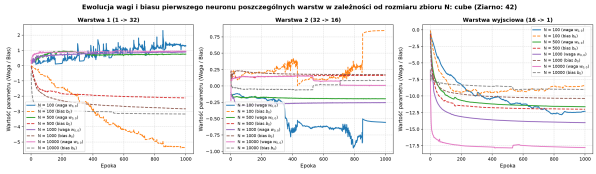
N	`data.activation`	`data.cube`
100	513.29 ± 221.95	90472.77 ± 2754.45
500	141.94 ± 1.62	76553.55 ± 289.43
1000	143.84 ± 6.20	77887.78 ± 1142.43
10000	243.46 ± 46.15	73660.82 ± 563.96

Po 1000 epokach nie zaobserwowano monotonicznej poprawy wraz ze wzrostem N. Dla `data.activation` najniższy testowy MSE uzyskano przy $N = 500$–1000; przy $N = 10000$ błąd wzrósł do 243.46, mimo bardzo małego błędu treningowego (0.011). Jest to oznaka pogorszenia generalizacji przy dłuższym treningu w tym ustawieniu. Dla `data.cube` większa próbka nadal pomagała ogólnie, ale wahania między $N = 500$ i $N = 1000$ pozostały. Testowy MSE wynosił 73660.82 dla $N = 10000$, a luka między błędem treningowym i testowym była duża.

Wykresy wagi $w_{0,0}$ i biasu b_0 w R4 (Rysunek 10) pokazują ich przebieg dla poszczególnych liczności zbioru uczącego. Są to parametry pierwszego neuronu każdej kolejnej warstwy dla reprezentatywnego ziarna 42.

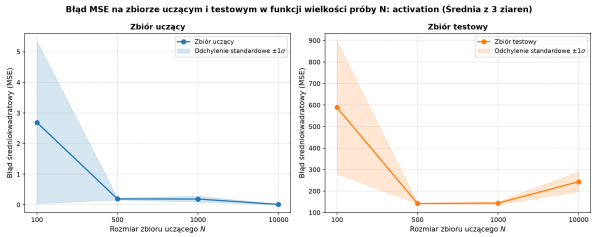


(a) `data.activation`.

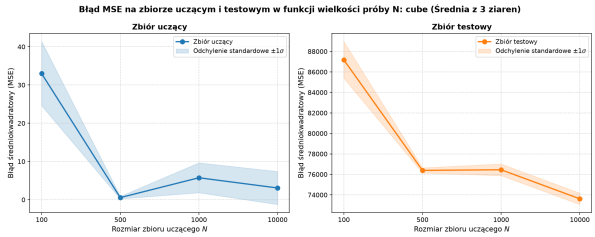


(b) `data.cube`.

Rysunek 10: Ewolucja wagi $w_{0,0}$ i biasu b_0 w kolejnych warstwach sieci R4 dla różnych liczności zbioru treningowego (ziarno 42, 1000 epok).



(a) `data.activation`



(b) `data.cube`

Rysunek 11: Błąd MSE na zbiorach uczących i testowych w zależności od liczności próby N. Wstęgi przedstawiają odchylenie standardowe dla trzech ziaren.

3.6 Podsumowanie i wnioski z części regresyjnej

Przeprowadzone eksperymenty pokazują, że wynik regresji MLP zależy łącznie od charakteru aproksymowanej funkcji, funkcji aktywacji, szerokości sieci, liczby próbek oraz współczynnika uczenia.

- Dla gładkiej funkcji `activation` aktywacje `Sigmoid` i `Tanh` zapewniły wysoką jakość aproksymacji, a poszerzenie jednej warstwy ukrytej z 4 do 64 neuronów znacząco obniżyło błąd. W badanych ustawieniach dodawanie kolejnych warstw nie przyniosło poprawy.

- Dla wielomianowego `cube` dobre dopasowanie w obszarze uczącym nie przełożyło się na ekstrapolację. Modele zwracały znacznie zaniżone predykcje poza przedziałem treningowym, co skutkowało wysokim błędem testowym i niskim R^2. Jest to ograniczenie testowanych modeli i konfiguracji, nie zaś dowód, że sieć MLP nie może aproksymować wielomianu w obszarze interpolacji.

- MSE osiągnęło najlepsze wyniki w porównaniu z MAE przy sprawdzonych współczynnikach uczenia. Eksperyment wskazał jednocześnie, że ocenę funkcji straty należy prowadzić wraz z dostrojeniem kroku uczenia.

- Zwiększenie liczby epok poprawiło część wyników (zwłaszcza w R2), ale nie wszystkie: w R1 wariant `ReLU` dla `cube` stał się niestabilny, a w R4 przy $N = 10000$ testowy MSE dla `activation` wzrósł. Liczba epok powinna zatem być dobierana na podstawie niezależnego zbioru walidacyjnego, a nie traktowana jako parametr, którego wzrost zawsze poprawia jakość.

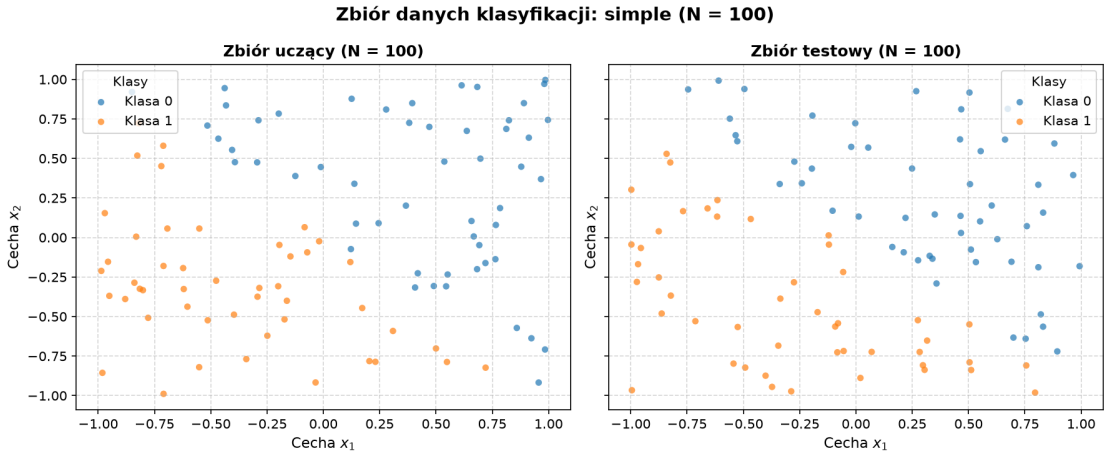
- Dane wykorzystano w surowej skali, zgodnie z implementacją eksperymentów. Standaryzacja cech i celów — wyznaczana na zbiorze uczącym i stosowana następnie do testowego — mogłaby ułatwić dobór współczynnika uczenia. Nie zmieniłaby jednak faktu, że zachowanie modelu daleko poza zakresem obserwacji wymaga osobnej oceny.

4 Eksperymenty — Zagadnienie klasyfikacji

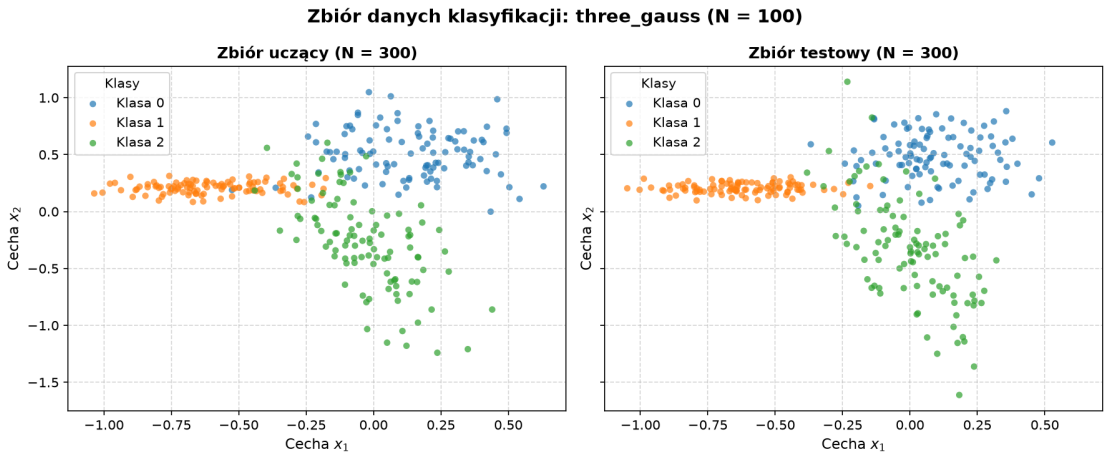
4.1 Zbiory danych

W tej części projektu, badano wykorzystanie modelu `MLP` do klasyfikacji punktów w przestrzeni $\mathbb{R}^2$. Do zadania dostarczono dwa zbiory danych, każdy w kilku wersjach różniących się wielkością. Aby uwypuklić różnice między różnymi architekturami, skupiono się na rozmiarze $N = 100$.

- **Zbiór `data.simple`** — klasyfikacja binarnej (2 klasy). Skupiska klas rozdzielone są łagodną, ale nieliniową granicą (Rysunek 12). Jeśli nie powiedziano wprost inaczej, w warstwie wyjściowej stosowano pojedynczy neuron z funkcją aktywacji `Sigmoid` oraz binarną entropię krzyżową (`BinaryCrossEntropyLoss`). Wyjątkiem jest eksperyment 3 którego celem było porównanie entropii krzyżowej do błędu średniokwadratowego (`MSE`).
- **Zbiór `data.three_gauss`** — klasyfikacja wieloklasowa (3 klasy). Skupiska powstały przez nałożenie na siebie dwuwymiarowych rozkładów gaussowskich (Rysunek 13). Kluczową cechą tego zbioru jest wyraźne nakładanie się rozkładów klas na środku przestrzeni. Ze względu na to przenikanie się klas, zbiór jest trudniejszy w klasyfikacji przez co idealne rozpoznawanie klas nie jest możliwe bez przeuczenia. Jeśli nie powiedziano wprost inaczej, na wyjściu sieci stosowano 3 neurony z aktywacją `Softmax`, etykiety kodowane w postaci one-hot oraz funkcję kategorycznej entropii krzyżowej (`CategoricalCrossEntropyLoss`). Wyjątkiem ponownie był eksperyment 3 w którym porównano entropię krzyżową z `MSE`.



Rysunek 12: Wizualizacja zbioru uczącego i testowego dla problemu binarnego `data.simple` ($N = 100$).



Rysunek 13: Zbiór uczący i testowy dla problemu wieloklasowego `data.three_gauss` ($N = 100$ na klasę). Widoczne nakładanie się klas w centrum przestrzeni.

4.2 Eksperyment 1 - Funkcje aktywacji w warstwach ukrytych

W ramach eksperymentu zbadano wpływ doboru funkcji aktywacji w warstwach ukrytych (`Sigmoid`, `Tanh`, `ReLU`) na przebieg uczenia, stabilność wag oraz finalny przebieg granic decyzyjnych. Eksperymenty przeprowadzono na jednej architekturze $[2, 32, 16, C]$ ($C = 1$ dla klasyfikacji binarnej, $C = 3$ dla wieloklasowej) przy użyciu optymalizatora `SGD` ($\eta = 0.05$, 150 epok) dla 10 różnych ziaren losowości.

Podsumowanie uzyskanych dokładności (*accuracy*) predykcji na zbiorze uczącym i testowym przedstawiono w Tabeli 5.

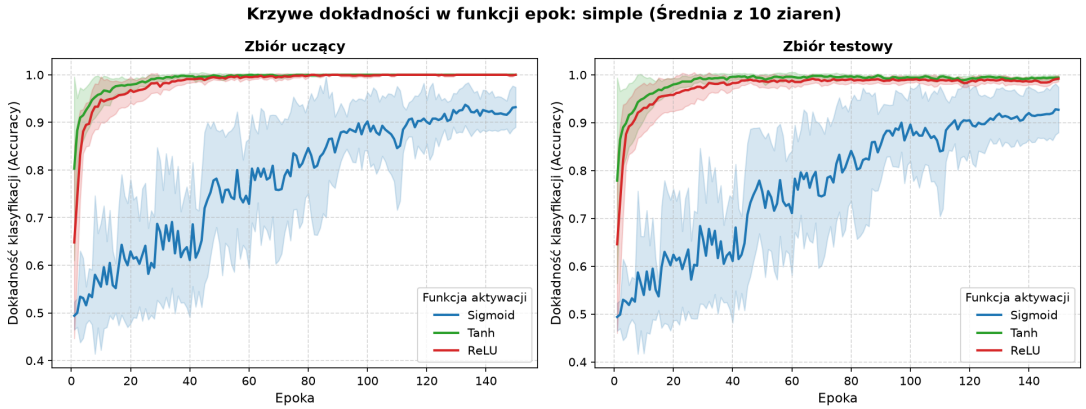
Tabela 5: Dokładność klasyfikacji dla modeli finalnych w zależności od funkcji aktywacji (średnia $\pm$ odchylenie standardowe z 10 ziaren).

Zbiór danych	Funkcja aktywacji	Zbiór testowy [%]	Zbiór uczący [%]
`data.simple`	Sigmoid	92.70 ± 4.75	93.20 ± 4.19
	Tanh	$\mathbf{99.40} \pm 0.49$	100.00 ± 0.00
	ReLU	99.20 ± 0.60	100.00 ± 0.00
`data.three_gauss`	Sigmoid	92.87 ± 0.22	92.37 ± 0.31
	Tanh	92.87 ± 0.16	92.63 ± 0.18
	ReLU	$\mathbf{93.17} \pm 0.27$	93.83 ± 0.48

4.2.1 Zbiór danych simple

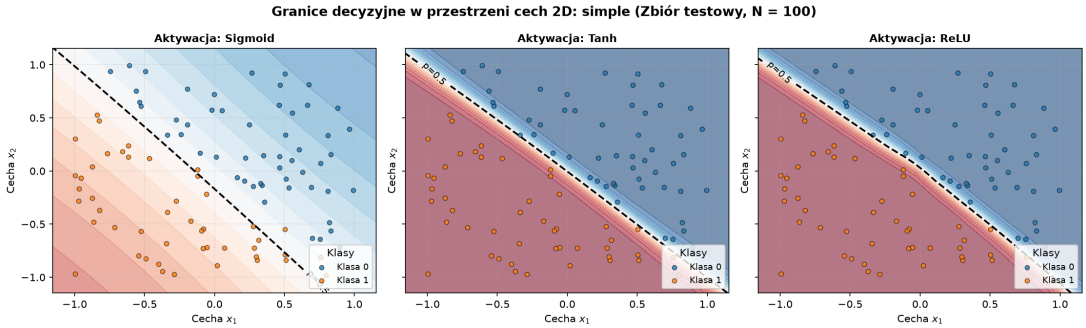
W zadaniu binarnym modele z aktywacją `Tanh` oraz `ReLU` zdołały w 100% nauczyć się rozpoznawać klasy na zbiorach treningowych i dobrze generalizują to na zbiorze testowym (99.40% dla `Tanh` oraz 99.20% dla `ReLU`). Funkcja `Sigmoid` zbiegała wyraźnie wolniej ze względu na nasycenie gradientu (stosunkowo mała wartość pochodnej funkcji sigma).

Na Rysunku 14 można zaobserwować, że dokładność sieci z aktywacją `Sigmoid` znacznie wolniej zbiega do 1. Dodatkowo trajektoria krzywej jest bardziej chaotyczna i wykazuje większe odchylenie standardowe (na badanej próbce 10 ziaren losowych), co sugeruje, że aktywacja `Sigmoid` jest bardziej wrażliwa na losową inicjalizację wag.



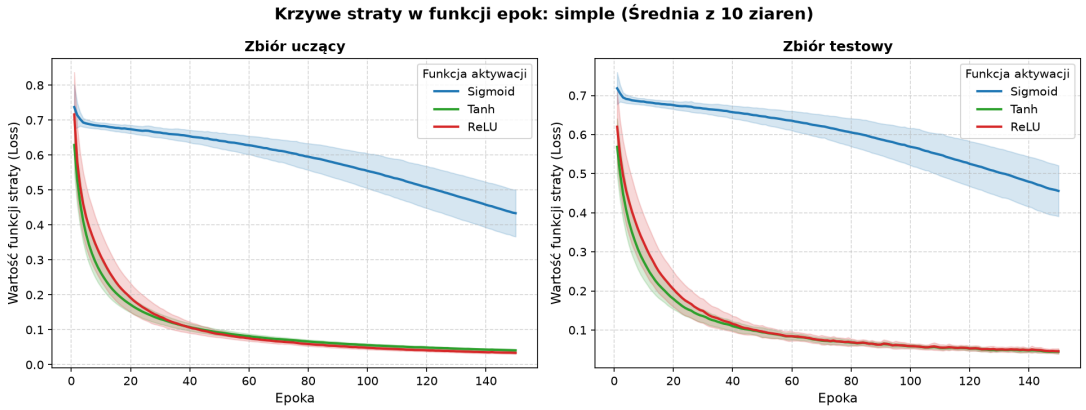
Rysunek 14: Przebieg dokładności klasyfikacji w kolejnych epokach dla zbioru `data.simple` (średnia z 10 ziaren z przedziałem $\pm 1\sigma$).

Gdy porównać nasycenie kolorów na Rysunku 15, widać, że granice decyzyjne dla funkcji `ReLU` i `Tanh` są jasno zdefiniowane, co oznacza, że model jest pewny swoich decyzji. W przypadku `Sigmoid` sytuacja jest odwrotna — występuje szerokie pasmo wysokiej niepewności (szacowane prawdopodobieństwo bliskie 0.5).



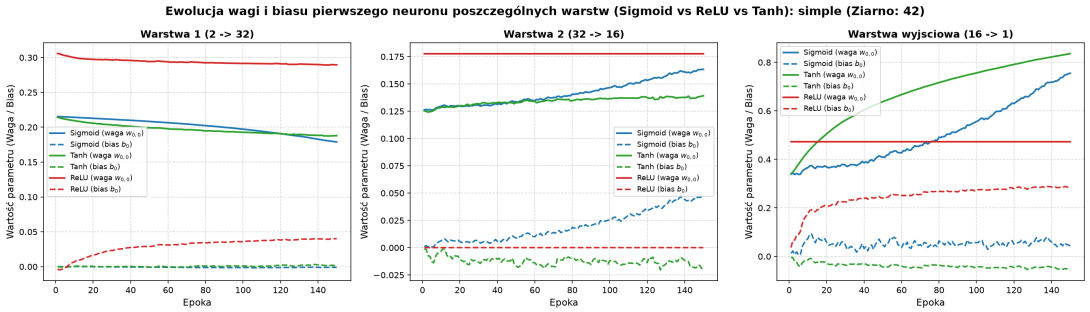
Rysunek 15: Granice decyzyjne w przestrzeni $\mathbb{R}^2$ dla zbioru `data.simple` w zależności od funkcji aktywacji.

Przebieg funkcji straty (Rysunek 16) pokazuje gwałtowny spadek błędu treningowego dla `ReLU` i `Tanh` już w pierwszych 20 epokach, podczas gdy strata w modelu `Sigmoid` maleje powoli i jednostajnie przez cały trening.



Rysunek 16: Przebieg funkcji straty w kolejnych epokach dla zbioru `data.simple` (średnia z 10 ziaren z przedziałem $\pm 1\sigma$).

Zmiany pierwszej wagi $w_{0,0}$ oraz biasu b_0 pierwszego neuronu (Rysunek 17) są stabilne i nie ma w nich oscylacji.

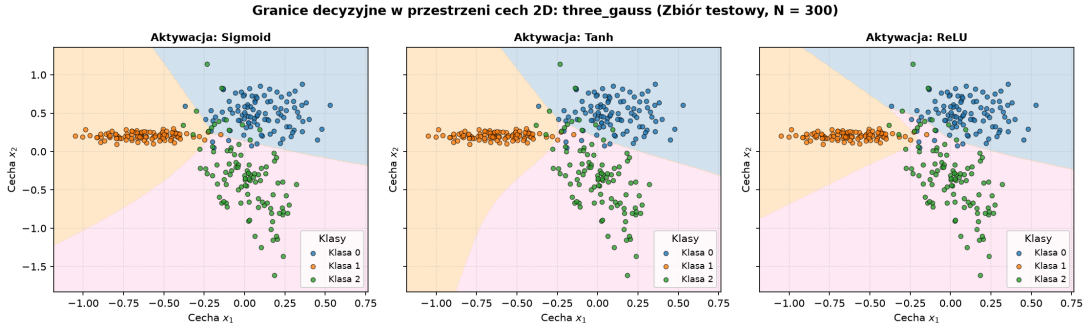


Rysunek 17: Ewolucja wagi $w_{0,0}$ i biasu b_0 pierwszego neuronu poszczególnych warstw dla zbioru `data.simple` (dla wybranego ziarna).

4.2.2 Zbiór danych three_gauss

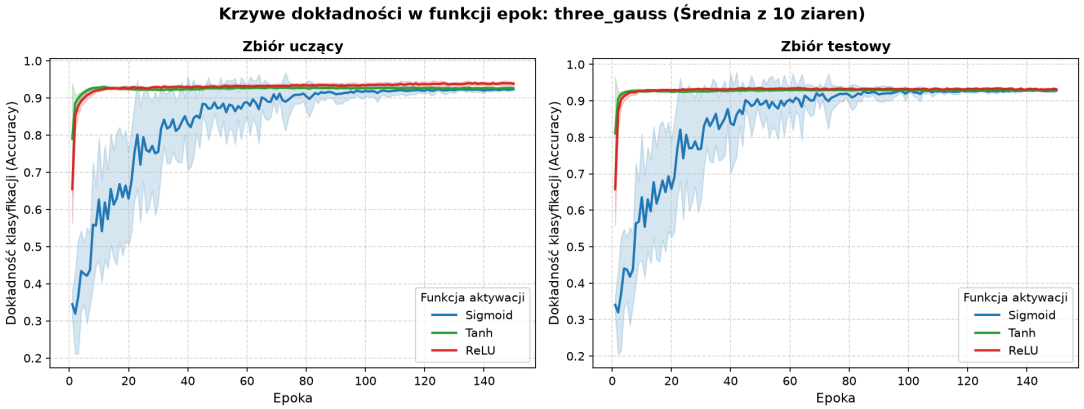
W zbiorze wieloklasowym wszystkie badane funkcje aktywacji osiągnęły zbliżoną dokładność na poziomie 92.9–93.2%. Ograniczenie to wynika z fizycznego nakładania się na siebie rozkładów trzech klas.

Na Rysunku 18 przedstawiono finalne granice decyzyjne w przestrzeni $\mathbb{R}^2$. Funkcja `ReLU` tworzy odcinkowo liniowe krawędzie podziału, podczas gdy `Tanh` i `Sigmoid` wyznaczają gładkie krzywe. We wszystkich modelach widoczne są punkty błędnie zaklasyfikowane w centralnej strefie nakładania się klas.



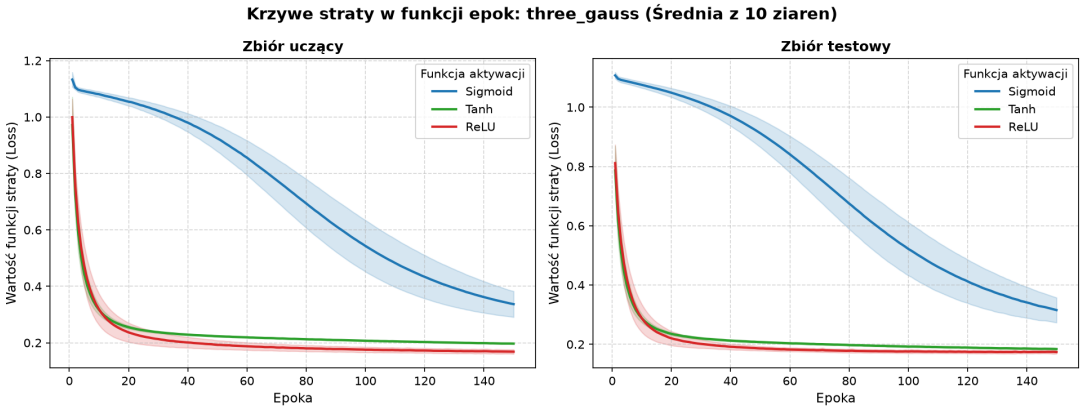
Rysunek 18: Granice decyzyjne w przestrzeni $\mathbb{R}^2$ dla zbioru `data.three_gauss` w zależności od funkcji aktywacji.

Krzywe dokładności na Rysunku 19 pokazują, że `ReLU` i `Tanh` już w kilkunastu epokach osiągają dokładność ok. 93%, podczas gdy `Sigmoid` początkowo mocno oscyluje i potrzebuje ponad 80 epok na ustabilizowanie predykcji.

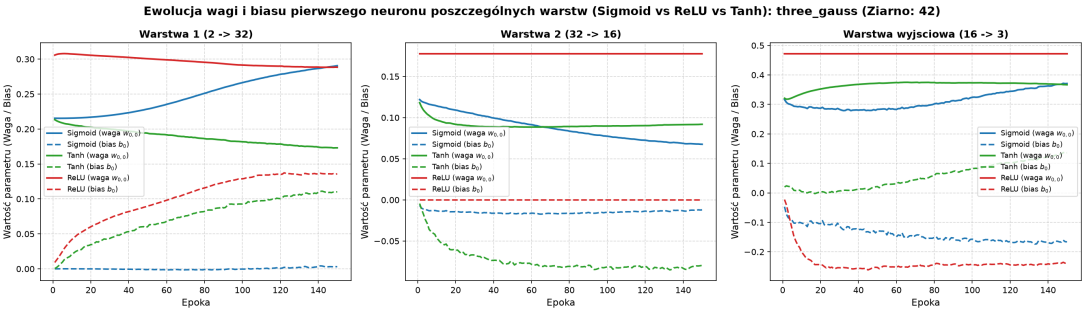


Rysunek 19: Przebieg dokładności klasyfikacji w kolejnych epokach dla zbioru `data.three_gauss` (średnia z 10 ziaren z przedziałem $\pm 1\sigma$).

Przebieg funkcji straty (Rysunek 20) oraz ewolucja parametrów pierwszego neuronu (Rysunek 21) obrazują analogiczne zjawiska szybkiej zbieżności i stabilności jak w przypadku zbioru `data.simple`.



Rysunek 20: Przebieg funkcji straty w kolejnych epokach dla zbioru `data.three_gauss` (średnia z 10 ziaren z przedziałem $\pm 1\sigma$).



Rysunek 21: Ewolucja wagi $w_{0,0}$ i biasu b_0 pierwszego neuronu poszczególnych warstw dla zbioru `data.three_gauss` (dla wybranego ziarna).

Wnioski. Podsumowując wyniki:

- Funkcje aktywacji `Tanh` oraz `ReLU` zapewniają znacznie szybszą redukcję błędu i wyższą odporność na losową inicjalizację parametrów niż funkcja `Sigmoid`, która cierpi na spowolnienie uczenia wywołane nasyceniem gradientu.

- Kształt wyznaczonych granic decyzyjnych odzwierciedla charakter funkcji aktywacji (odcinkowo liniowe dla `ReLU` vs gładkie dla `Tanh`). W zbiorze `data.three_gauss` w którym klasy się nakładają wszystkie modele osiągają zbliżoną skuteczność ograniczoną górną granicą ok. 93%.
- Zmiana wyszczególnionych wag i biasów w sieci przebiega stabilnie we wszystkich wariantach, potwierdzając że implementacja jest poprawna. W kolejnych eksperymentach te wykresy będą pominietę dla większej czytelności.

4.3 Eksperyment 2 - Architektura sieci

W ramach eksperymentu zbadano przebieg uczenia i finalne wyniki sieci w zależności od jej głębokości (od 0 do 4 warstw ukrytych) oraz wielkości poszczególnych warstw. Przetestowano 8 wariantów (gdzie C oznacza liczbę rozpoznawanych klas):

- model liniowy bez warstw ukrytych $[2, C]$
- trzy modele z 1 warstwą ukrytą ($[2, 4, C]$, $[2, 16, C]$, $[2, 64, C]$)
- dwa modele 2 warstwowe ($[2, 16, 8, C]$, $[2, 32, 16, C]$)
- sieć 3 warstwową $[2, 32, 16, 8, C]$
- sieć 4 warstwową $[2, 64, 32, 16, 8, C]$

Na podstawie wyników eskperymentu 1, tym razem zastosowano wszędzie funkcję aktywacji `ReLU` w warstwach ukrytych. Podobnie jak w poprzednim eksperymencie, wykorzystano 10 różnych ziaren losowości. Natomiast hiperparametry uczenia zostały lekko zmodyfikowane ($\eta = 0.01$, 250 epok) aby głębsze sieci miały szansę zostać dotrenowane.

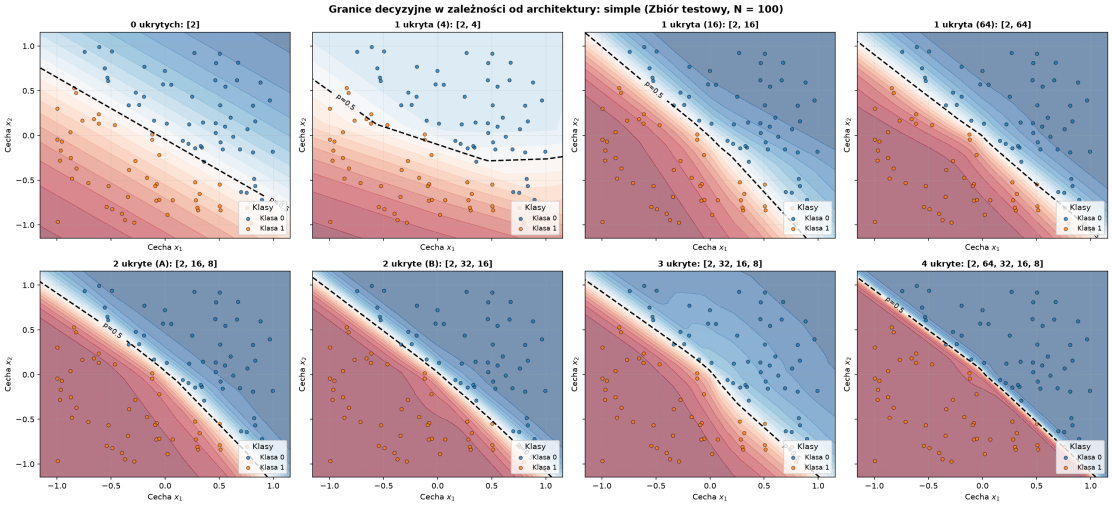
4.3.1 Zbiór danych simple

Wyniki dokładności klasyfikacji dla badanych architektur na zbiorze `data.simple` przedstawiono w Tabeli 6.

Tabela 6: Dokładność klasyfikacji w zależności od architektury dla zbioru `data.simple` (średnia ± odchylenie standardowe z 10 ziaren).

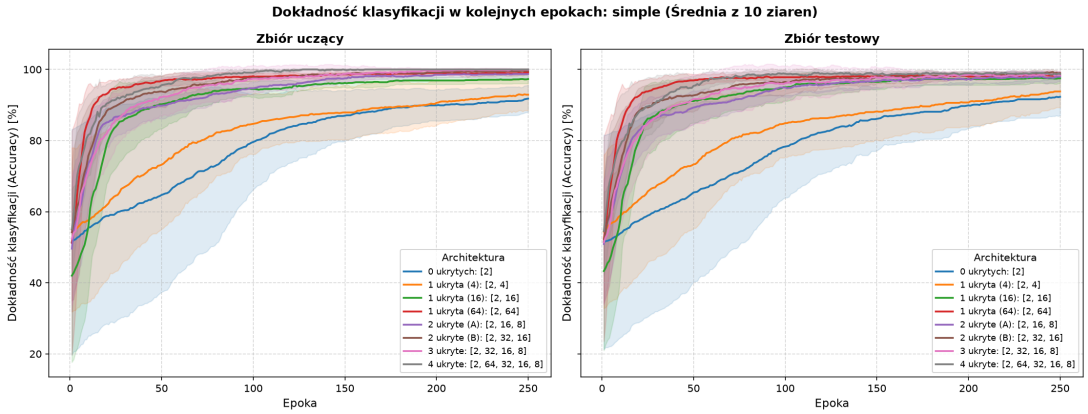
Architektura	**Warstwy ukryte**	**Zbiór testowy [%]**	**Zbiór uczący [%]**
0 ukrytych: [2]	0	92.30 ± 5.25	91.80 ± 3.82
1 ukryta (4): [2, 4]	1	93.80 ± 4.49	92.90 ± 4.44
1 ukryta (16): [2, 16]	1	97.50 ± 1.43	97.30 ± 1.27
1 ukryta (64): [2, 64]	1	98.50 ± 0.81	99.20 ± 0.60
2 ukryte (A): [2, 16, 8]	2	98.00 ± 1.34	98.70 ± 1.19
2 ukryte (B): [2, 32, 16]	2	$\mathbf{99.00 \pm 0.63}$	99.60 ± 0.66
3 ukryte: [2, 32, 16, 8]	3	98.40 ± 1.11	99.90 ± 0.30
4 ukryte: [2, 64, 32, 16, 8]	4	98.70 ± 0.46	100.00 ± 0.00

Na Rysunku 22 przedstawiono zmiany granic decyzyjnych wraz ze wzrostem złożoności sieci. Model liniowy (zero warstw ukrytych) wyznacza liniową granicę, która nie jest w stanie dokładnie odwzorować zakrzywionego kształtu klas podczas gdy dodanie jednej ukrytej warstwy 16 neuronów pozwala na płynne dopasowanie granicy i znacznie lepsze podzielenie przestrzeni. Głębsze architektury oferują bardziej pewne predykcje i bardziej szczegółową krzywą rozdzielającą przestrzeń.



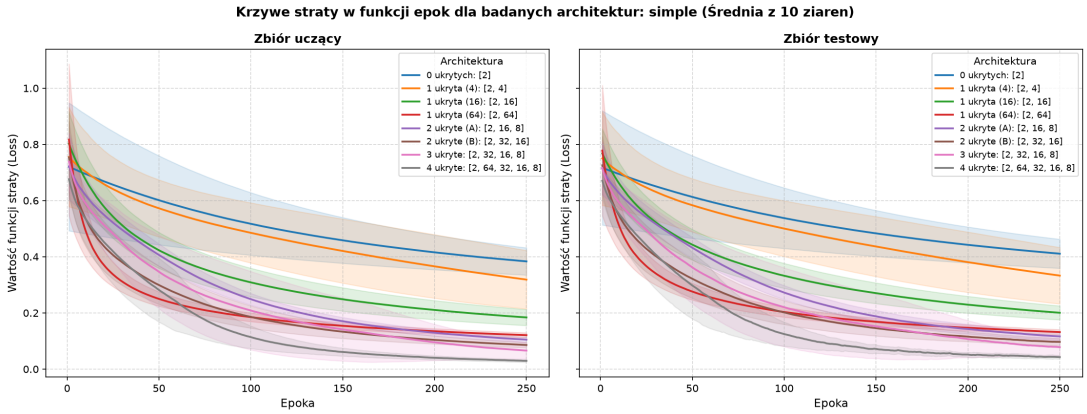
Rysunek 22: Kształt granic decyzyjnych w przestrzeni cech $\mathbb{R}^2$ dla zbioru `data.simple` w zależności od architektury sieci.

Krzywe dokładności wraz z postępami w uczeniu (Rysunek 23) pokazują, że sieci głębsze (2-4 warstwy) osiągają zbieżność znacznie szybciej i mają mniejsze odchylenie standardowe. Wąska sieć $[2, 4]$ uczy się z kolei bardzo wolno, z szerokim przedziałem niepewności co sugeruje że jest wrażliwa na inicjalizację wag.



Rysunek 23: Przebieg dokładności klasyfikacji w kolejnych epokach dla zbioru `data.simple` w zależności od architektury (średnia z 10 ziaren z przedziałem $\pm 1\sigma$).

Podobnie w przypadku funkcji straty (Rysunek 24), sieci płytkie uczą się wolniej, a dodawanie kolejnych warstw z reguły sprawia że sieć szybciej zmniejsza swój błąd uczenia.



Rysunek 24: Przebieg funkcji straty w kolejnych epokach dla zbioru `data.simple` w zależności od architektury (średnia z 10 ziaren z przedziałem $\pm 1\sigma$).

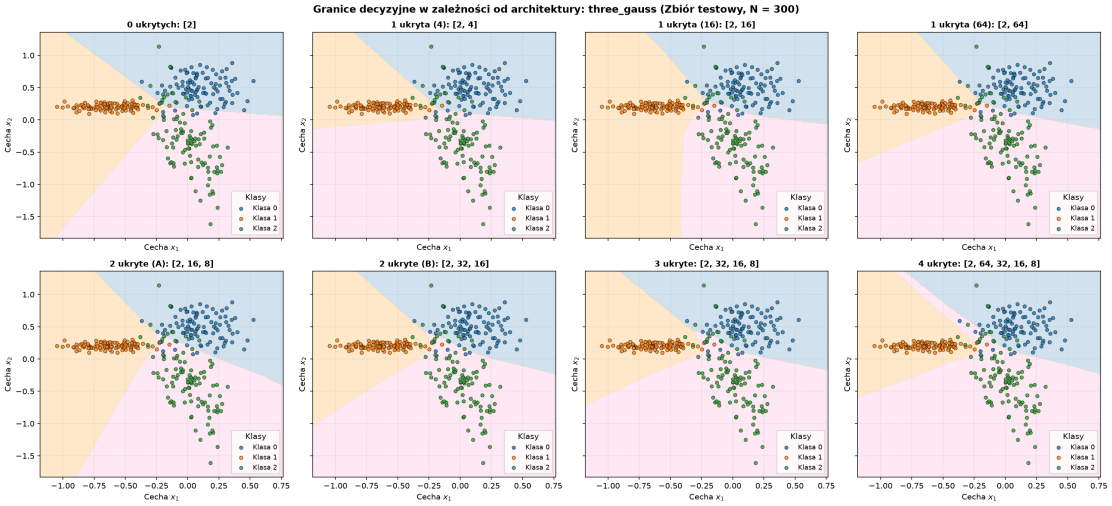
4.3.2 Zbiór danych three_gauss

Wyniki dokładności klasyfikacji w zbiorze wieloklasowym `data.three_gauss` zestawiono w Tabeli 7. Tak jak w eksperymencie 1, można zaobserwować że żaden z modeli nie przekroczył granicy ok. 93.0% z uwagi na nakładające się klasy. Natomiast modele głębsze radziły sobie odrobinę lepiej. Ciekawe jest to, że liniowy model bez żadnych warstw ukrytych osiąga podobny wynik do pozostałych sieci. Wskazuje to na to, że zbiór jest dość łatwy do rozdzielenia w sposób liniowy, a nauczenie się szumu wynikającego z nakładania się klas nie pozwala głębszym modelom osiągnąć znacząco lepszych wyników.

Tabela 7: Dokładność klasyfikacji w zależności od architektury dla zbioru `data.three_gauss` (średnia $\pm$ odchylenie standardowe z 10 ziaren).

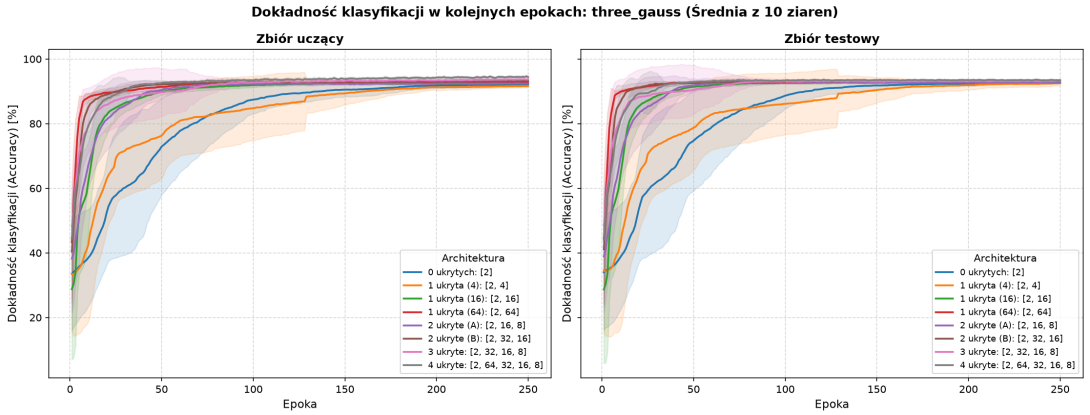
Architektura	**Warstwy ukryte**	**Zbiór testowy [%]**	**Zbiór uczący [%]**
0 ukrytych: [2]	0	92.70 ± 0.38	92.20 ± 0.65
1 ukryta (4): $[2, 4]$	1	92.53 ± 0.78	91.63 ± 0.41
1 ukryta (16): $[2, 16]$	1	92.73 ± 0.29	92.40 ± 0.44
1 ukryta (64): $[2, 64]$	1	93.07 ± 0.25	92.73 ± 0.33
2 ukryte (A): $[2, 16, 8]$	2	92.90 ± 0.33	92.47 ± 0.34
2 ukryte (B): $[2, 32, 16]$	2	$\mathbf{93.43 \pm 0.33}$	93.20 ± 0.60
3 ukryte: $[2, 32, 16, 8]$	3	93.23 ± 0.21	93.77 ± 0.67
4 ukryte: $[2, 64, 32, 16, 8]$	4	93.40 ± 0.29	94.43 ± 0.33

Na Rysunku 25 zobrazowano granice decyzyjne wyznaczone przez badane architektury. Model liniowy, podobnie jak wcześniej dzieli przestrzeń prostymi, podczas gdy architektury głębsze tworzą dokładniejsze granice klasyfikacji. We wszystkich przypadkach jednak w centrum obrazka widoczne są punkty błędnie zaklasyfikowane z uwagi na nakładanie się rozkładów.

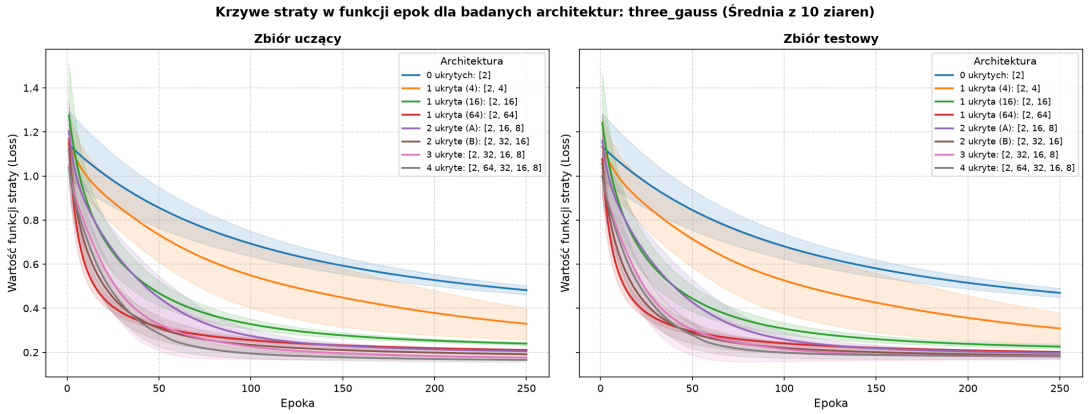


Rysunek 25: Kształt granic decyzyjnych w przestrzeni cech $\mathbb{R}^2$ dla zbioru `data.three_gauss` w zależności od architektury sieci.

Przebieg dokładności klasyfikacji (Rysunek 26) oraz funkcji straty (Rysunek 27) w kolejnych epokach prezentują zbieżne tendencje jak dla problemu binarnego: modele głębsze szybciej redukują stratę i osiągają stabilniejszą dokładność.



Rysunek 26: Przebieg dokładności klasyfikacji w kolejnych epokach dla zbioru `data.three_gauss` w zależności od architektury (średnia z 10 ziaren z przedziałem $\pm 1\sigma$).



Rysunek 27: Przebieg funkcji straty w kolejnych epokach dla zbioru `data.three_gauss` w zależności od architektury (średnia z 10 ziaren z przedziałem $\pm 1\sigma$).

Wnioski. Podsumowując badanie wpływu głębokości i szerokości modelu `MLP`:

- W tych konkretnych zadaniach, model bez warstw ukrytych (liniowy) nieco gorzej od sieci głębszych. Dla zbiory `data.simple` podział jest na tyle prosty, że model liniowy nie popełnia wielu błędów. Z kolei dla `data.three_gauss` szum jest na tyle duży że modele głębokie nie mają szansy daleko prześcignąć modelu liniowego, mimo że uczą się szybciej i osiągają mniejszy błąd.
- Zwiększenie głębokości sieci do 2-4 warstw znacząco przyspiesza tempo zbieżności i obniża błąd treningowy, ale przyrost dokładności powyżej 2 warstw ukrytych jest już mniejszy. Dobrym kompromisem między wielkością sieci i osiąganą dokładnością wydaje się architektura średnia $[2, 32, 16]$. Osiąga ona najwyższy wynik na obu zbiorach (99.00% dla `data.simple` oraz 93.43% dla `data.three_gauss`).
- W zbiorze `data.simple` modele głębokie osiągają niemal zerowe straty treningową bez widocznego spadku wyników na zbiorze testowym. Do tego odchylenia standardowe metryk wskazują na względną stabilność metody `SGD` względem ziarna losowego.

4.4 Eksperyment 3 - Wpływ funkcji straty

W tym eksperymencie zbadano wpływ doboru funkcji straty na proces uczenia oraz jakość klasyfikacji. Porównano dwie funkcje celu: entropię krzyżową (`Cross-Entropy`: `BinaryCrossEntropy` + aktywacja wyjściowa `Sigmoid` dla klasyfikacji binarnej oraz `CategoricalCrossEntropy` + aktywacja wyjściowa `Softmax` dla wieloklasowej) oraz błąd średniokwadratowy (`MSE` z aktywacją wyjściową odpowiednią dla zbioru danych). Pozostałe elementy sieci ustalono na podstawie wyników dwóch poprzednich eksperymentów:

- architektura $[2, 32, 16, C]$, gdzie $C = 1$ dla klasyfikacji binarnej i $C = 3$ dla wieloklasowej
- warstwy ukryte z aktywacją `ReLU` i inicjalizacją `He`

Aby dokładniej sprawdzić zachowanie miar błedu, przetestowano trzy wartości współczynnika uczenia: $\eta \in \{0.01, 0.05, 0.2\}$. Wyniki uśredniono z 10 różnych ziaren losowości.

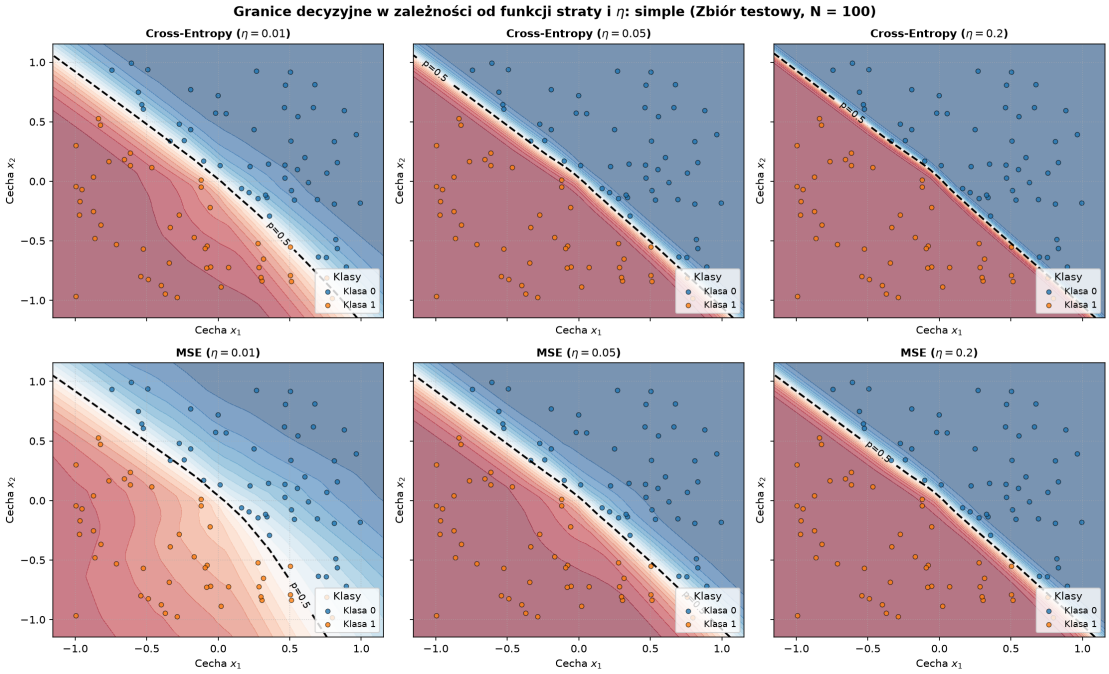
4.4.1 Zbiór danych simple

Wyniki dokładności klasyfikacji na zbiorze binarnym `data.simple` zestawiono w Tabeli 8. `Cross-Entropy` osiąga wysoką dokładność już przy najniższym kroku uczenia $\eta = 0.01$ (97.90%). W przypadku błędu średniokwadratowego (`MSE`) przy małym kroku $\eta = 0.01$ sieć radzi sobie zauważalnie słabiej (95.20%). Dopiero zwiększenie kroku uczenia do $\eta = 0.05$ lub $\eta = 0.2$ pozwala modelowi z funkcją `MSE` dorównać wynikom `Cross-Entropy`.

Tabela 8: Dokładność klasyfikacji w zależności od funkcji straty i współczynnika uczenia η dla zbioru `data.simple` (średnia $\pm$ odchylenie standardowe z 10 ziaren).

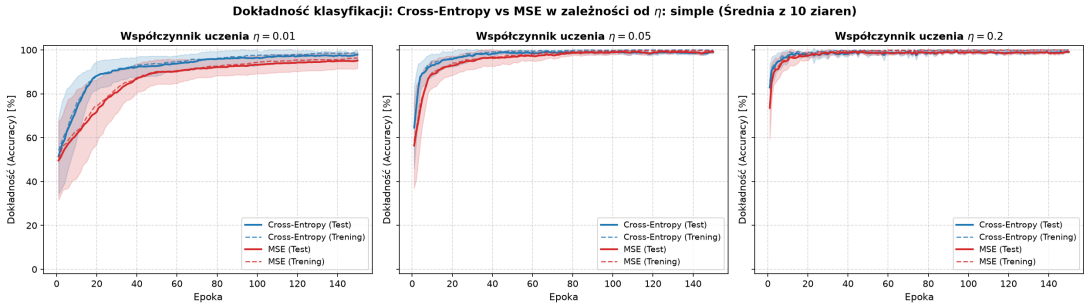
Funkcja straty	**Krok uczenia (η)**	**Zbiór testowy [%]**	**Zbiór uczący [%]**
`Cross-Entropy`	0.01	97.90 ± 1.87	98.70 ± 1.35
`Cross-Entropy`	0.05	$\mathbf{99.20 \pm 0.60}$	100.00 ± 0.00
`Cross-Entropy`	0.20	99.10 ± 0.54	100.00 ± 0.00
`MSE`	0.01	95.20 ± 3.54	96.30 ± 3.03
`MSE`	0.05	99.10 ± 0.83	99.80 ± 0.40
`MSE`	0.20	99.10 ± 0.54	100.00 ± 0.00

Na Rysunku 28 przedstawiono wyznaczone granice decyzyjne. Obie funkcje straty poprawnie rozdzielają zbiór, jednak dla `MSE` przy $\eta = 0.01$ granica jest mniej gładka i widać większe niezdecydowanie modelu w pobliżu punktów skrajnych. Przy wyższych współczynnikach uczenia granice stają się ostre i bardzo pewne w obu przypadkach.

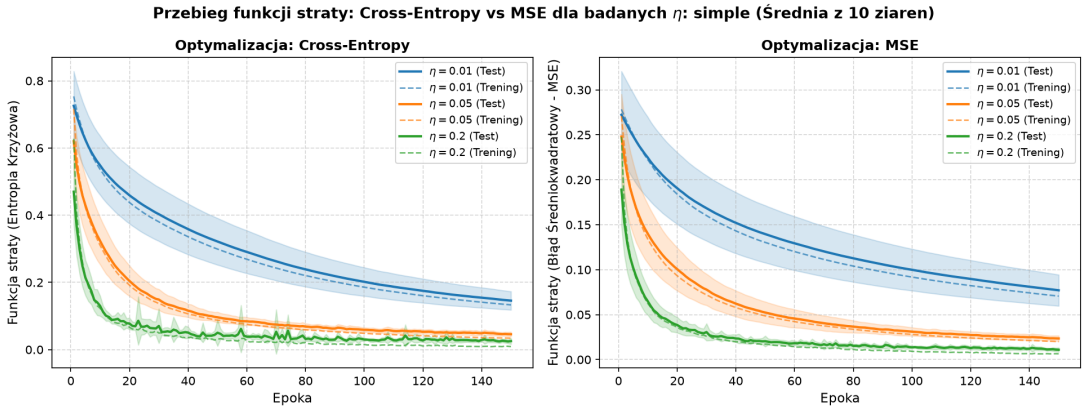


Rysunek 28: Granice decyzyjne w zależności od funkcji straty i kroku uczenia η dla zbioru `data.simple`.

Przebieg dokładności (Rysunek 29) oraz straty (Rysunek 30) ujawnia wyraźną przewagę tempa uczenia na korzyść `Cross-Entropy`. Model z `Cross-Entropy` osiąga niemal maksymalną dokładność już w ciągu pierwszych kilkunastu epok, podczas gdy `MSE` przy $\eta = 0.01$ uczy się wolno i z większym rozrzutem pomiędzy ziarnami.



Rysunek 29: Przebieg dokładności klasyfikacji w kolejnych epokach dla funkcji `Cross-Entropy` oraz `MSE` przy różnych wartościach η (`data.simple`, średnia z 10 ziaren z przedziałem $\pm 1\sigma$).



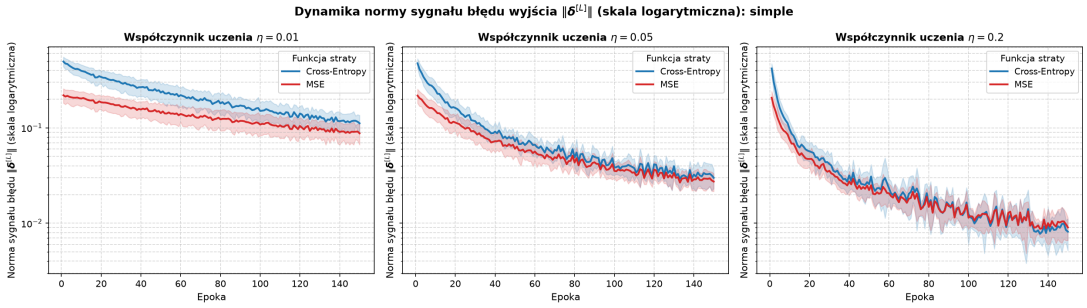
Rysunek 30: Przebieg funkcji straty w kolejnych epokach dla `Cross-Entropy` i `MSE` na rozdzielonych skalach (`data.simple`, średnia z 10 ziaren z przedziałem $\pm 1\sigma$).

Różnice w dynamice uczenia wynikają bezpośrednio z postaci początkowego sygnału błędu $\boldsymbol{\delta}^{[L]}$ generowanego na wyjściu sieci (Rysunek 31). Na wykresie wartości na osi rzędnych (osi Y) przedstawiono w skali logarytmicznej, co pozwala uwidocznić różnice rzędów wielkości w sile propagowanego gradientu. W procedurze propagacji wstecznej zmiana funkcji straty i aktywacji wyjściowej wpływa wyłącznie na początkowy sygnał błędu $\boldsymbol{\delta}^{[L]}$, podczas gdy pozostała część propagacji wstecznej (aktualizacja wag $\partial\mathcal{L}/\partial\boldsymbol{W}^{[l]} = \boldsymbol{\delta}^{[l]}(\boldsymbol{a}^{[l-1]})^T$ oraz sygnały warstw ukrytych) pozostaje nienaruszona.

W zależności od doboru funkcji celu sygnał błędu na wyjściu przyjmuje postać:

$$\begin{aligned} \text{Cross-Entropy}: &\quad \boldsymbol{\delta}^{[L]} = \hat{\boldsymbol{y}} - \boldsymbol{y}, \\ \text{MSE}: &\quad \boldsymbol{\delta}^{[L]} = (\hat{\boldsymbol{y}} - \boldsymbol{y}) \odot \hat{\boldsymbol{y}} \odot (\mathbf{1} - \hat{\boldsymbol{y}}). \end{aligned}$$

Dla błędu średniokwadratowego (`MSE`) w sygnale pojawia się nasycający czynnik $\hat{\boldsymbol{y}} \odot (\mathbf{1} - \hat{\boldsymbol{y}})$. Gdy prawdopodobieństwo wyjściowe $\hat{\boldsymbol{y}}$ zbliża się do 0 lub 1, czynnik ten dąży do zera, co drastycznie tłumi gradient o rzędy wielkości (co widać na wykresie w skali log). Dla `Cross-Entropy` zjawisko nasycenia nie występuje, a wielkość błędu zależy wprost od różnicy $(\hat{\boldsymbol{y}} - \boldsymbol{y})$, co zapewnia stabilne i dynamiczne uczenie nawet przy małym kroku $\eta = 0.01$.



Rysunek 31: Przebieg normy sygnału błędu warstwy wyjściowej $\|\boldsymbol{\delta}^{[L]}\|$ w kolejnych epokach treningu na zbiorze `data.simple` (oś Y w skali logarytmicznej, średnia z 10 ziaren z przedziałem $\pm 1\sigma$). Widoczne tłumienie gradientu o rzędy wielkości dla `MSE` wskutek obecności czynnika nasycenia $\hat{\boldsymbol{y}} \odot (\mathbf{1} - \hat{\boldsymbol{y}})$.

4.4.2 Zbiór danych three_gauss

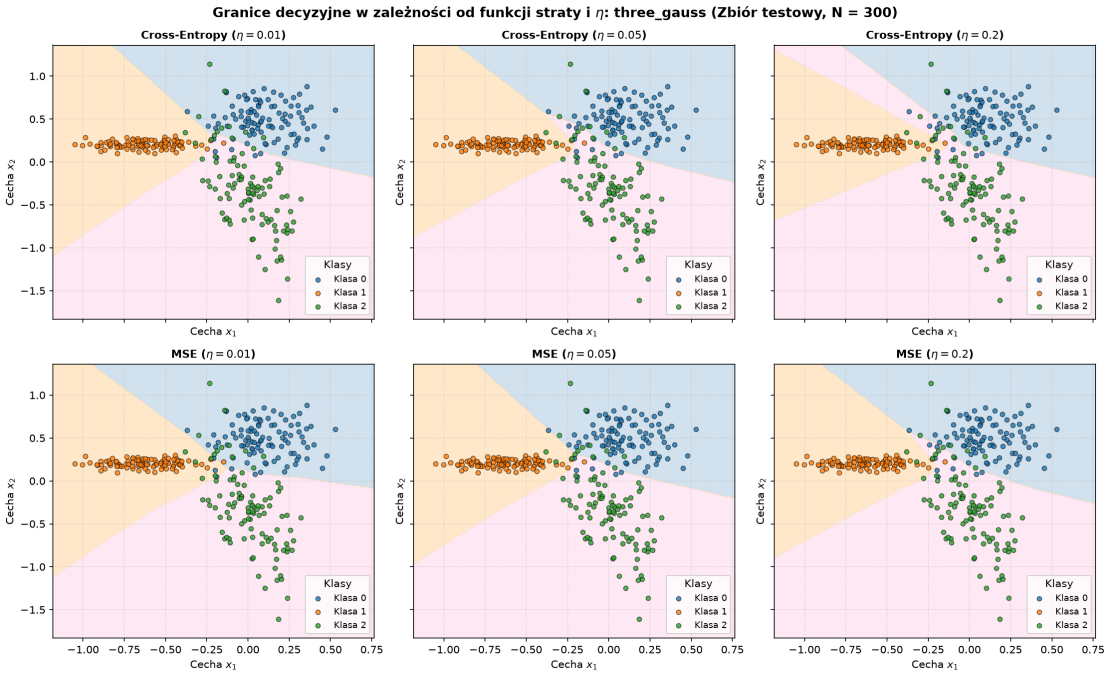
Wyniki dokładności dla wieloklasowego zbioru `data.three_gauss` z warstwą wyjściową `Softmax` przedstawiono w Tabeli 9. Tak jak w poprzednich eksperymentach, ze względu na fizyczne nakładanie się rozkładów gaussowskich, żaden model nie przekracza progu ok. 93.2%.

Nową obserwacją jest to, że w problemie wieloklasowym różnice w końcowej dokładności pomiędzy `Cross-Entropy` a `MSE` są jeszcze mniejsze: zarówno `Cross-Entropy`, jak i `MSE` osiągają ok. $93.0 - 93.2\%$ na zbiorze testowym. Niemniej jednak, przy małym kroku $\eta = 0.01$ funkcja `MSE` uczy się zauważalnie wolniej na zbiorze treningowym (91.40% wobec 92.97% dla `Cross-Entropy`), co wskazuje na występowanie analogicznego zjawiska spłaszczenia gradientu przez pochodną funkcji `Softmax`.

Tabela 9: Dokładność klasyfikacji w zależności od funkcji straty i współczynnika uczenia η dla zbioru `data.three_gauss` (średnia $\pm$ odchylenie standardowe z 10 ziaren).

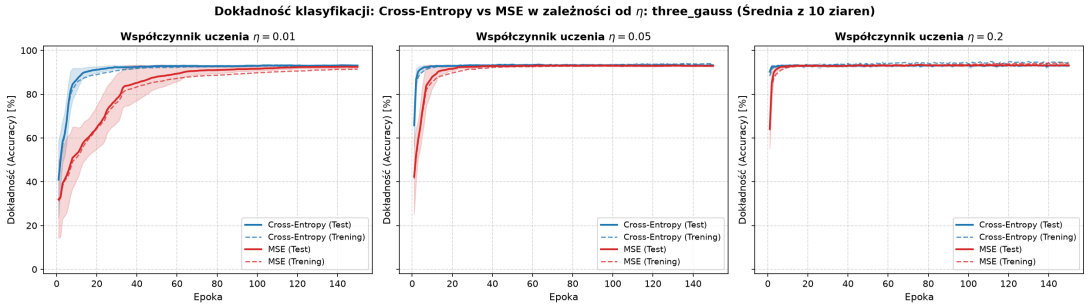
Funkcja straty	**Krok uczenia (η)**	**Zbiór testowy [%]**	**Zbiór uczący [%]**
`Cross-Entropy`	0.01	93.10 ± 0.26	92.97 ± 0.35
`Cross-Entropy`	0.05	$\mathbf{93.17 \pm 0.27}$	93.83 ± 0.48
`Cross-Entropy`	0.20	93.10 ± 0.33	94.70 ± 0.81
`MSE`	0.01	92.50 ± 0.69	91.40 ± 0.96
`MSE`	0.05	92.97 ± 0.43	92.97 ± 0.38
`MSE`	0.20	93.10 ± 0.26	93.93 ± 0.44

Na Rysunku 32 zobrazowano granice decyzyjne dla badanych wariantów. Dla obu funkcji straty przy krokach $\eta = 0.05$ i $\eta = 0.2$ granice prawidłowo wyodrębniają trzy obszary, a punkty błędnie zaklasyfikowane leżą wyłącznie w rejonie wzajemnego przenikania się klas w centrum.

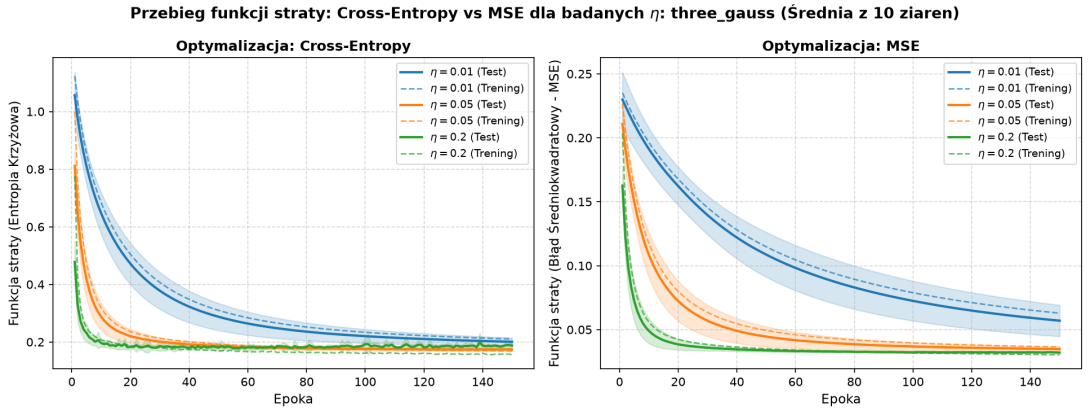


Rysunek 32: Granice decyzyjne w zależności od funkcji straty i kroku uczenia η dla zbioru `data.three_gauss`.

Przebieg dokładności klasyfikacji (Rysunek 33) oraz funkcji straty (Rysunek 34) w problemie wieloklasowym wykazuje zbieżne tendencje: `MSE` przy małym kroku $\eta = 0.01$ uczy się wolniej, lecz przy $\eta \geq 0.05$ osiąga skuteczność porównywalną z `Cross-Entropy`.

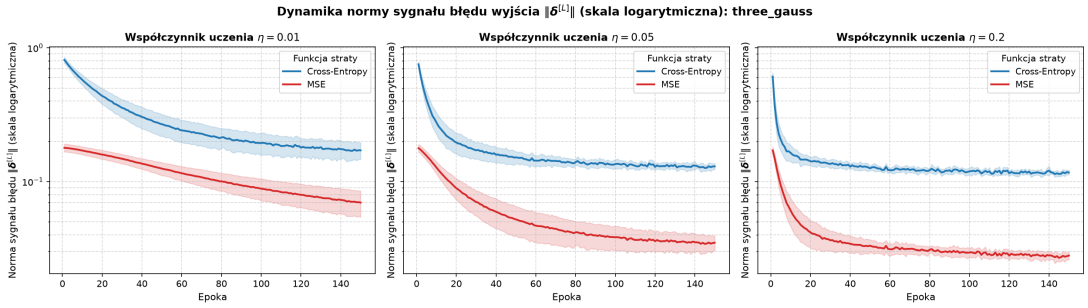


Rysunek 33: Przebieg dokładności klasyfikacji w kolejnych epokach dla zbioru `data.three_gauss` (średnia z 10 ziaren z przedziałem $\pm 1\sigma$).



Rysunek 34: Przebieg funkcji straty w kolejnych epokach dla zbioru `data.three_gauss` (średnia z 10 ziaren z przedziałem $\pm 1\sigma$).

Dynamika normy sygnału błędu wyjścia $\|\boldsymbol{\delta}^{[L]}\|$ przedstawiona na Rysunku 35 (w skali logarytmicznej) potwierdza analogiczne tłumienie gradientu wyjściowego dla funkcji `MSE` w początkowych epokach treningu na zbiorze wieloklasowym.



Rysunek 35: Przebieg normy sygnału błędu warstwy wyjściowej $\|\boldsymbol{\delta}^{[L]}\|$ w kolejnych epokach treningu na zbiorze `data.three_gauss` (oś Y w skali logarytmicznej, średnia z 10 ziaren z przedziałem $\pm 1\sigma$).

Wnioski. Podsumowując porównanie `Cross-Entropy` oraz błędu średniokwadratowego (`MSE`) w zadaniach klasyfikacji:

- **Przewaga `Cross-Entropy`**: Entropia krzyżowa jest lepiej dopasowana do zadań klasyfikacyjnych. Zapewnia szybszą zbieżność i wysoką dokładność już przy małych wartościach kroku uczenia ($\eta = 0.01$).

- **Zjawisko nasycenia w `MSE`**: Zastosowanie funkcji `MSE` wprowadza do początkowego sygnału błędu $\boldsymbol{\delta}^{[L]}$ czynnik nasycenia $\hat{\boldsymbol{y}} \odot (\mathbf{1} - \hat{\boldsymbol{y}})$. Gdy predykcja wyjściowa dąży do 0 lub 1, gradient ulega stłumieniu o rzędy wielkości (co uwidoczniono na wykresach w skali logarytmicznej), drastycznie spowalniając początkowy etap uczenia przy małym kroku η.
- **Wpływ współczynnika uczenia**: Wolniejsze uczenie modelu z funkcją `MSE` można częściowo skompensować poprzez zwiększenie wartości η (np. do 0.05 lub 0.2). Przy większym kroku uczenia sieć z funkcją `MSE` osiąga ostatecznie zbliżoną dokładność do modelu z `Cross-Entropy`. Mimo to, entropia krzyżowa ma tę przewagę że nie musimy ręcznie dostrajać współczynnika uczenia.

4.5 Podsumowanie i wnioski z części klasyfikacyjnej

- **Funkcja aktywacji**: Najlepsze rezultaty zapewnia funkcja `ReLU` (w połączeniu z inicjalizacją `He` - zgodnie z [1]). Zabezpiecza ona model przed zanikaniem gradientu i gwarantuje najszybszą zbieżność w porównaniu do nasycającej funkcji funkcji `Sigmoid`.
- **Architektura sieci**: Zwiększenie liczby warstw ukrytych przyspiesza proces uczenia i pozwala wyznaczać dokładniejsze granice decyzyjne. Najlepszym kompromisem pomiędzy rozmiarem sieci a dokładnością okazała się architektura 2-warstwowa ($[2, 32, 16]$) — dalsze pogłębianie modelu przynosiło niewielki zysk na zbiorze testowym.
- **Funkcja straty**: Entropia krzyżowa (`Cross-Entropy`) jest lepszym wyborem niż `MSE` dla problemu klasyfikacji. Eliminuje problem nasycenia gradientu na wyjściu z aktywacją `Sigmoid` lub `Softmax`, co zapewnia szybkie i stabilne uczenie już przy małym kroku η.

5 Podsumowanie ogólne i wnioski końcowe

Literatura

[1] The Deep Hub, *He and Xavier Weight Initialization Functions*, `https://medium.com/thedeephub/he-and-xavier-weight-initialization-functions-acedc5322ce5`.